

1 · Definition of Reinforcement Learning

Formal Definition (Sutton & Barto)

Reinforcement Learning is learning **what to do** — how to map situations to actions — so as to **maximise a numerical reward signal**.

The learner is not told which actions to take. It must discover which actions yield the most reward by **trying them** (trial and error).

Actions may affect not only the immediate reward but also the next situation — and through that, **all subsequent rewards** (delayed consequences).

Three distinguishing features of RL:

1. **Trial and error** — no labelled dataset; the agent generates its own experience.
2. **Delayed reward** — consequences of an action may not be felt until much later.
3. **Active interaction** — the agent's actions change the environment and future inputs.

2 · Elements of Reinforcement Learning

The 6 Core Elements

Element	Meaning
Agent	The learner / decision-maker.
Environment	Everything outside the agent; the world the agent interacts with.
State s	A representation of the current situation (what the agent observes).
Action a	A choice available to the agent in a given state.
Reward R_t	A scalar feedback signal from the environment after each action.
Policy π	A mapping from states to actions (the agent's behaviour).
Value function $V(s)$	Expected cumulative future reward from state s under a policy.
Model (optional)	The agent's internal model of how the environment works.

The RL loop (one step):

Agent observes S_t $\rightarrow$ chooses A_t $\rightarrow$ Environment returns R_{t+1}, S_{t+1} $\rightarrow$ agent updates

Reward vs Value function:

- **Reward R_t** is *immediate* — what you get right now.
- **Value $V(s)$** is *long-term* — the total expected reward from this point onward.
- Reward is given by the environment; value is *estimated* by the agent.
- All RL methods are fundamentally about **efficiently estimating value functions**.

Policy vs Model:

- **Policy:** directly maps state $\rightarrow$ action (what to do).
- **Model:** predicts what the next state and reward will be if a certain action is taken (*model-based* methods use this; *model-free* methods do not).

3 · How RL Differs from Machine Learning

Three-Way Comparison

Property	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Training signal	Labelled examples (correct output given)	No labels	Scalar reward (delayed, noisy)
Data source	Static dataset	Static dataset	Agent's own experience
Goal	Generalise to new inputs	Find structure/clusters	Maximise cumulative reward
Actions change data?	No	No	Yes — actions change future observations
Who decides best action?	Teacher (label)	N/A	Agent discovers it via trial & error

Key exam point: In RL there is no *supervisor* giving the correct action. The agent only gets a reward signal — which may be sparse, delayed, or noisy. This is fundamentally different from supervised learning where the correct answer is provided directly.

4 · Tic-Tac-Toe as an RL Problem

Mapping Tic-Tac-Toe to RL Elements

RL Element	Tic-Tac-Toe equivalent
State s	The current board configuration (which squares are X, O, or empty).
Action a	Placing your mark (X or O) in an empty square.
Reward R	+1 (win), -1 (lose), 0 (draw or non-terminal).
Policy π	The rule used to choose which square to play next.
Value $V(s)$	Estimated probability of winning from board position s .
Environment	The opponent + game rules.

Initialising the Value Function

At the start of training (before any games are played):

$$V(s) = \begin{cases} 1.0 & \text{if } s \text{ is a position where we have already won} \\ 0.0 & \text{if } s \text{ is a position where we have already lost or drawn} \\ 0.5 & \text{for all other positions (uncertain — could go either way)} \end{cases}$$

How the Agent Plays and Learns

1. **Move selection:** Most of the time, pick the move with the **highest** $V(s)$ (greedy — exploit). Occasionally, pick a **random** move (explore, the ϵ -greedy idea).
2. **After each greedy move:** update the value of the state just left using the **Temporal Difference (TD) equation**.

5 · The Temporal Difference (TD) Update Equation

TD Update — The Central Equation of Session 1

$$V(S_t) \leftarrow V(S_t) + \alpha [V(S_{t+1}) - V(S_t)]$$

Every term explained:

- $V(S_t)$ — current value estimate of the state we just left
- $V(S_{t+1})$ — current value estimate of the state we just entered
- α — learning rate (step size), $0 < \alpha \leq 1$
- $[V(S_{t+1}) - V(S_t)]$ — the **TD error**: how surprised we were
- The whole expression: nudge $V(S_t)$ slightly toward $V(S_{t+1})$

Why $V(S_{t+1})$ and not the actual reward? In the middle of a game, there is no immediate reward — just a board position. We use the *next state's estimated value* as a proxy for the true value. As $V(S_{t+1})$ itself improves through experience, $V(S_t)$ also improves. This is called **bootstrapping**: using estimates to improve estimates.

Worked Example — TD Update

Given: $V(S_t) = 0.5$ (current state looks 50-50), $\alpha = 0.1$.

Case 1: We move to a strong position, $V(S_{t+1}) = 0.8$

$$\text{TD error} = 0.8 - 0.5 = 0.3$$

$$V(S_t) \leftarrow 0.5 + 0.1 \times 0.3 = 0.5 + 0.03 = \mathbf{0.53}$$

The current state looks slightly better than we thought.

Case 2: We move to a weak position, $V(S_{t+1}) = 0.2$

$$\text{TD error} = 0.2 - 0.5 = -0.3$$

$$V(S_t) \leftarrow 0.5 + 0.1 \times (-0.3) = 0.5 - 0.03 = \mathbf{0.47}$$

The current state looks slightly worse than we thought.

Role of α (Learning Rate)

α setting	Effect	Use case
$\alpha \rightarrow 0$ over time	Converges to true value (stationary env.)	Theorem: guaranteed convergence if $\sum \alpha_t = \infty$ and $\sum \alpha_t^2 < \infty$
Reduced but never 0	Continues adapting; won't fully converge	Good practical choice for stationary tasks
α constant	Tracks changes; never fully converges	Non-stationary environments (opponent changes)

Tic-Tac-Toe: RL vs Other Approaches

Approach	Key idea	Limitation compared to RL
Minimax	Exhaustive game tree search; assumes perfect opponent	Does not <i>learn</i> — same policy regardless of opponent; not suited for imperfect/unknown opponents
Genetic / Evolutionary	Evaluate many policies; select and breed best performers	No temporal credit assignment; slow feedback; many games needed per policy; wasteful
RL (TD)	Learn a value function by playing; improve incrementally	Requires many games to converge; initial estimates affect learning path

6 · Key Takeaways for the Exam

Must-Know Points — Session 1

1. **RL definition:** learning by trial and error to maximise cumulative reward. No supervisor; only a reward signal.
2. **Elements:** agent, environment, state, action, reward, policy, value function (model is optional). Know what each means.
3. **RL vs SL:** No labelled data; agent generates experience; actions change future inputs.
4. **TD equation:** $V(S_t) \leftarrow V(S_t) + \alpha[V(S_{t+1}) - V(S_t)]$. Know every symbol and what the update does.
5. **Tic-Tac-Toe initialisation:** won=1.0, lost/drawn=0.0, unknown=0.5.
6. **Bootstrapping:** value estimates are updated using *other value estimates* (not just final rewards).
7. **α effect:** large α = fast but noisy; small/decaying α = stable but slow; constant α = good for non-stationary.

DRL EC3 Exam Handout — Session 1 (Introduction to RL). Ref: Sutton & Barto Ch. 1.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27